

Pestを導入して機能を活用しよう ～主にCI最適化を例に～

2026/09/24 第58回関西PHP勉強会
おさない

このセッションで持ち帰って欲しいこと

- Pestは検証・導入を容易にできる
 - PHPUnitのコードベースのまま実行環境だけをPestにすることも可能
- テストを最適化するための多様な機能が公式から提供
 - PHPUnit記法のまま使える拡張も多数
 - CIの最適化以外にも「便利な機能」が公式から提供

※しゃべらないこと：Pestの詳細な記法について等

Pestとは

- "The elegant testing framework for PHP developers and AI agents."
 - 関数ベースの記法、expectation API
- <https://pestphp.com/>

```
// Pest
it('performs sums', function () {
    expect(sum(1, 2))->toBe(3);
});
```

```
// PHPUnit
public function test_performs_sums()
{
    $this->assertSame(3, sum(1, 2));
}
```

Pestの特徴

1. 自然言語に近い構文
2. PHPUnitからの段階的移行が可能
3. テストを便利にする多様な機能が本体や公式プラグインに内包

※このセッションは2,3について触れます

PHPUnitからの段階的移行

- Pest上でPHPUnit構文のテストを実行できる
 - *Pest is built on top of PHPUnit*
- PHPUnit → Pest への移行を補助する公式プラグイン
 - `pestphp/pest-plugin-drift`

テストを便利にする機能が本体や公式プラグインに内包

- テスト最適化: `--parallel` / `--shard` / `--profile` / `--compact`
- コード品質: Mutation Testing (`--mutate`), Architecture Testing (`arch()`) など
- 拡張: Laravel / PHPStan / Rector / E2E など

Pest導入からCI最適化機能活用までの手順

1. Pestをエンジンとして導入する
2. CIを `./vendor/bin/pest --ci` に差し替える
3. 最適化機能を足す

1. Pestの導入

- <https://pestphp.com/docs/installation>
- 最新のPest 5は PHP 8.4 以上が必要

1. Pestの導入

- 依存関係の導入

```
composer remove phpunit/phpunit  
composer require pestphp/pest --dev --with-all-dependencies
```

- Pestの初期化、実行

- PHPUnitの既存テストケースはそのまま実行可能

```
./vendor/bin/pest --init  
./vendor/bin/pest
```

2. CIのコマンドの置き換え

- Continuous Integration > Exampleを参照に.ymlを追加
 - <https://pestphp.com/docs/continuous-integration>
- GHAでPHPUnitを置き換える場合、`run` を置き換える

PHPUnit

```
- name: Tests  
  run: ./vendor/bin/phpunit
```

Pest

```
- name: Tests  
  run: ./vendor/bin/pest --ci
```

3. Pest テスト最適化機能の活用

- テストを最適化するための機能がPestに内包
 - <https://pestphp.com/docs/optimizing-tests>

並列化する

- └ Parallel Testing
- └ Test Sharding (Time-Balanced Sharding)

ボトルネックを把握する

- └ Profiling

失敗に集中する

- └ Compact Printer

Parallel Testing

- 同じ実行環境上で、テストを複数のプロセスで並列実行させる
- PHPUnitにおけるParaTestのイメージ
- テストの実行時間を短縮できる

```
./vendor/bin/pest --parallel
```

Test A → Test B → Test C → Test D

↓

Test A ┌
Test B ┤ → 同時実行
Test C ┤
Test D └

Test Sharding

- CIでのテスト実行時、テストを複数のCIジョブに分割
- GitHub ActionsのMatrixのようなイメージ
- テストの実行時間を短縮できる可能性がある

```
./vendor/bin/pest --shard=1/4
```

Test Suite → CI #1 (1/4) CI #2 (2/4) CI #3 (3/4) CI #4 (4/4)

Time-Balanced Sharding

- デフォルトの `--shard` はテストファイル数で分割する
 - 特定のファイルに遅いテストがあると、テスト全体実行完了が遅くなる
- 過去の実行時間でCIジョブを分割するTime-Balanced Sharding
 - CIジョブのボトルネックを最小限に抑えることができる

```
./vendor/bin/pest --update-shards  
./vendor/bin/pest --shard=1/4
```

`tests/.pest/shards.json` をコミットすると、`--shard` は時間ベースになる

Profiling

- 実行時間を計測し、時間がかかっているテストを表示する
- 遅いテストを特定、最適化の手がかりになる

```
./vendor/bin/pest --profile
```

```
Tests:      100 passed (153 assertions)
```

```
Duration:   11.68s
```

```
Top 10 slowest tests:
```

```
Tests\Feature\UserTest > create user 6.27s
```

```
Tests\Feature\OrderTest > create order 4.91s
```

```
Tests\Feature\ProductTest > create product 0.24s
```

```
...
```

```
(98.88% of 11.68s) 11.55s
```

Compact Printer

- 失敗したテストだけを表示させる
- 出力が単純になるので、I/Oが減り数ミリ秒速くなることもある

```
./vendor/bin/pest --compact
```

```
→ ./vendor/bin/pest --compact
```

```
× .....  
.....
```

```
FAILED Tests\Unit\ExampleTest > that true is true  
Failed asserting that true is false.
```

```
at tests/Unit/ExampleTest.php:4  
<?php
```

組み合わせるとこうなる

4ジョブに分ける（それぞれ 1/4～4/4）

```
./vendor/bin/pest --ci --parallel --compact --shard=1/4
```

- ※ ジョブごとに番号を変える。1/4 の1ジョブだけではスイートの 4 分の 1
- ※ `--update-shards` は記録用。毎回は付けない
- ※ `--profile` は診断用。日常CIには付けない

補足 Parallel,shardにおける注意点

- テストが分離・独立していないと失敗する場合がある
 - テスト間でのデータベースや外部リソースの競合がある場合
 - テストが実行順序に依存している場合
- 小規模なテストだとオーバーヘッドで逆に遅くなることも

補足 TIAモード

- TIA：変更で壊れ得るテストだけを再実行する
 - Pest5の目玉機能
 - The TIA engine is a game-changer for test speeds. Laravel Cloud's test suite of 19,000+ tests went from 3 minutes to 5 seconds:
- 下記2点の理由からTIAは本セッションのスコープ外
 - CI上では--tiaモードを使わないことを強く推奨
 - --tiaモードの利用にはpestのfunctional記法への書き換えが必要

まとめ

- Pestは検証・導入を容易にできる
- テストを最適化するための多様な機能が公式から提供
- 「次は、Pestの導入から、始まる」

PHPカンファレンス関西2026開催

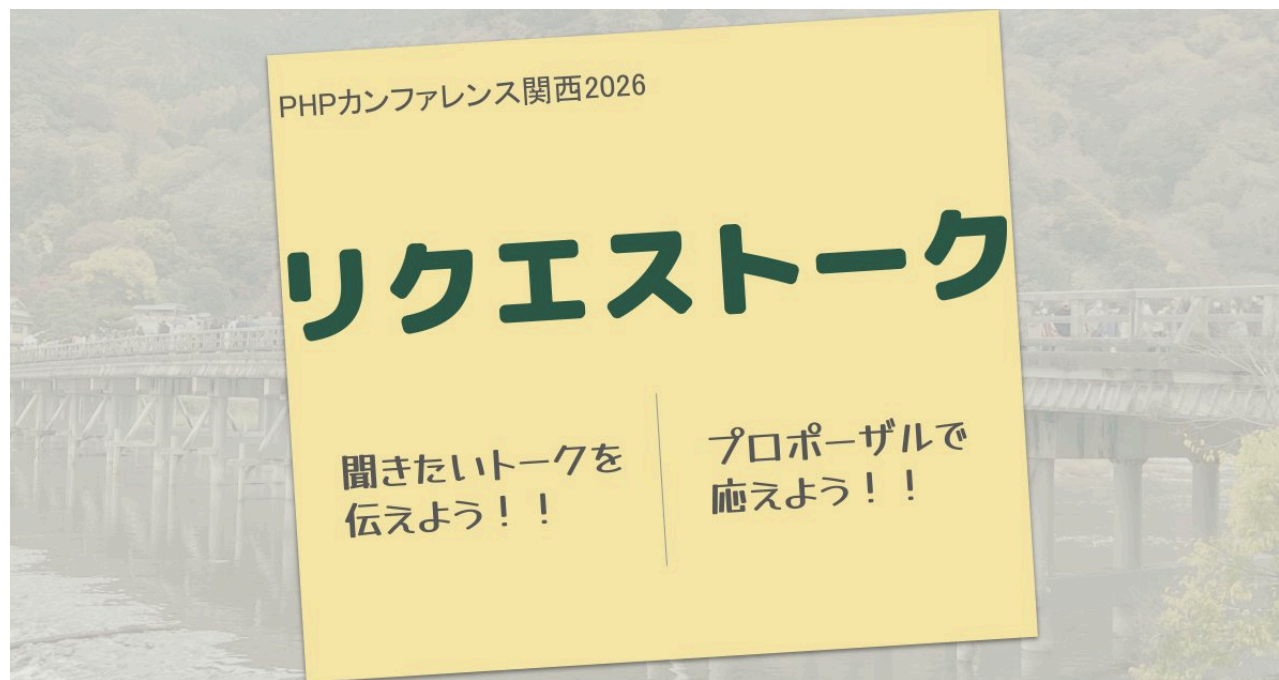
- スポンサー募集中！
- プロポーザル、リクエストトーク絶賛募集中！



<https://2026.kphpug.jp/>

新企画「リクエストーク」

- PHPカンファレンス関西2026で「聞きたいトーク」を聞かせてください
- 「聞きたい！」をプロポーザル出したい方にお届けします！
- 「新たなコミュニティへの携わり方」を作ります！



今年もプロポーザル作成ワークショップを行います！

- 参加者募集開始！
- <https://phpkansai.connpass.com/event/407721/>



自己紹介



おさない

Twitter: @000sak000

PHPカンファレンス愛媛2026

- 2026/10/03(土) 愛媛県松山市で開催！
- チケット発売中！
- （ここで、実行委員長のひがきさんからのありがたいお話）



<https://phpcon.ehime.jp/>

PHPカンファレンス新潟2026

- 2026/11/14(土) 新潟県新潟市で開催！
- チケット販売中！当日スタッフ募集中！



<https://phpcon.niigata.jp/>

phper x 日本酒の会 #8 in 関西

- 2026/10/23(土) 大阪で開催！
- 参加者募集中！
- <https://phponshu.connpass.com/event/406131/>

